

**THE COLLEGE OF
TECHNOLOGY (COLTECH)**

THE UNIVERSITY OF BAMENDA



**DEPARTMENT OF
COMPUTER ENGINEERING**

REPUBLIC OF CAMEROON

*A Dissertation Submitted to the Department of Computer Engineering in the
College of Technology (COLTECH) of the University of Bamenda in Partial
Fulfillment of the Requirements for the Award of a MTech - Master of
Technology Degree in Computer Engineering.*

BY:

AUTHOR NAME

REGISTRATION NUMBER: -

SUPERVISOR(S):

-

COPYRIGHT

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without prior written permission of the author or The University of Bamenda.

DECLARATION OF ORIGINALITY OF STUDY

I, AUTHOR NAME, registration N° : -, in the Department of COMPUTER ENGINEERING in the College of Technology (COLTECH) of The University of Bamenda hereby declare that this work titled "REPUBLIC OF CAMEROON" is my original work. It has not been presented in any application for a degree or any academic pursuit. I have acknowledged all borrowed ideas nationally and internationally through citations.

Date: _____ Signature of author _____

CERTIFICATION OF CORRECTIONS AFTER DEFENSE

This is to certify that this dissertation titled "REPUBLIC OF CAMEROON" is the original work of AUTHOR NAME. This work is submitted in partial fulfillment of the requirements for the award of a MTech - Master of Technology Degree in COMPUTER ENGINEERING in the College of Technology (COLTECH) of The University of Bamenda.

Supervisor _____ :

Pr. SUPERVISOR NAME

The _____ Head _____ of _____ Department _____ :

Pr. HEAD OF DEPARTMENT NAME

The _____ Director _____ :

Pr. DIRECTOR NAME

ACCEPTANCE OF DISSERTATION

Having met the stipulated requirements, this dissertation entitled "REPUBLIC OF CAMEROON" has been accepted by the Postgraduate School of The University of Bamenda in partial fulfillment of the requirements for the award of a MTech - Master of Technology Degree in COMPUTER ENGINEERING (Specialty : COMPUTER ENGINEERING).

Chairperson, Master's Dissertation Committee : _____

Pr. SUPERVISOR NAME

Date : _____

ABSTRACT

This Dissertation presents an in-depth investigation into REPUBLIC OF CAMEROON. The study formulates the core architecture, evaluates performance metrics, and establishes practical implementation guidelines in accordance with institutional standards.

RÉSUMÉ

Ce travail présente une étude approfondie sur republic of cameroon. L'étude formule l'architecture principale, évalue les métriques de performance et établit des directives d'application pratiques conformément aux normes institutionnelles.

DEDICATION

Dedicated to my family, supervisors, and colleagues whose encouragement and support made this academic pursuit possible.

ACKNOWLEDGEMENTS

I express my sincere gratitude to my supervisor, the department faculty, and all individuals who contributed advice, guidance, and assistance throughout the execution of this work.

TABLE OF CONTENTS

COPYRIGHT	i
DECLARATION OF ORIGINALITY OF STUDY	ii
CERTIFICATION	iii
ACCEPTANCE OF DISSERTATION	iv
ABSTRACT	v
RÉSUMÉ	vi
DEDICATION	vii
ACKNOWLEDGEMENTS	viii
LIST OF TABLES	xi
LIST OF FIGURES	xii
LIST OF ABBREVIATIONS	xiii
CHAPTER 1: INTRODUCTION	1
1.1 INTRODUCTION	1
CHAPTER 2: LITERATURE REVIEW	2
2.1 LITERATURE REVIEW	2
CHAPTER 3: MATERIALS AND METHODS (PROPOSED METHODOLOGY) ..	3
3.1 Proposed Work Plan	10
3.2 Background	3
3.3 Description of the Research Problem	4
3.4 Research Questions	4
3.5 Objectives	4
1.4 Rationale	5
3.6 Justification and Motivation	5
3.7 Significance	5
3.8 Scope	5

3.9 Limitation	6
3.10 Conceptual Overview	6
3.11 Classical File Integrity Checking Approaches	6
3.12 Host-Based Intrusion Detection and Real-Time Monitoring	7
3.13 Challenges: False Positives and Tamper Resistance	7
3.14 Compliance-Driven Requirements	8
3.15 Research Gap	8
3.16 Research Design	8
3.17 Proposed System Architecture Overview	9
3.18 Baseline and Hashing Strategy	10
3.19 Tools and Technologies	10
3.20 Detection and Alerting Logic	10
3.21 Evaluation Metrics	11
3.22 Ethical Considerations	11
3.23 Proposed Work Plan	11
REFERENCES	11

LIST OF TABLES

Table 1.1: 1.1 Background	1	1.2 Description of the Research Problem
Table 2.1: 2.1 Conceptual Overview . . .	5	2.2 Classical File Integrity Checking Approaches
Table 3.1: REFERENCES	11	
Table 3.2: Summary Table	11	

LIST OF FIGURES

No figures detected in this document.

LIST OF ABBREVIATIONS

FIM	File Integrity Monitoring
SHA	Secure Hash Algorithm
API	Application Programming Interface
OS	Operating System
SIEM	Security Information and Event Management
DSR	Design Science Research

CHAPTER 1: INTRODUCTION

1.1 INTRODUCTION

Table 1.1: 1.1 Background 1 1.2 Description of the Research Problem 2 1.3 Research Questions and Objectives 2 1.4 Rationale 3 1.5 Scope and Limitation 3

1.1 Background	1
1.2 Description of the Research Problem	2
1.3 Research Questions and Objectives	2
1.4 Rationale	3
1.5 Scope and Limitation	3

CHAPTER 2: LITERATURE REVIEW

2.1 LITERATURE REVIEW

Table 2.1: 2.1 Conceptual Overview 5 2.2 Classical File Integrity Checking Approaches 5 2.3 Host-Based Intrusion Detection and Real-Time Monitoring 5 2.4 Challenges: False Positives and Tamper Resistance 6 2.5 Compliance-Driven Requirements 6 2.6 Research Gap 6 CHAPTER 3: MATERIALS AND METHODS (PROPOSED METHODOLOGY) 8 3.1 Research Design 8 3.2 Proposed System Architecture Overview 8 3.3 Baseline and Hashing Strategy 9 3.4 Tools and Technologies 9 3.5 Detection and Alerting Logic 9 3.6 Evaluation Metrics 10 3.7 Ethical Considerations 10

2.1 Conceptual Overview	5
2.2 Classical File Integrity Checking Approaches	5
2.3 Host-Based Intrusion Detection and Real-Time Monitoring	5
2.4 Challenges: False Positives and Tamper Resistance	6
2.5 Compliance-Driven Requirements	6
2.6 Research Gap	6
CHAPTER 3: MATERIALS AND METHODS (PROPOSED METHODOLOGY)	8
3.1 Research Design	8
3.2 Proposed System Architecture Overview	8
3.3 Baseline and Hashing Strategy	9
3.4 Tools and Technologies	9
3.5 Detection and Alerting Logic	9
3.6 Evaluation Metrics	10
3.7 Ethical Considerations	10

CHAPTER 3: MATERIALS AND METHODS (PROPOSED METHODOLOGY)

3.1 Proposed Work Plan 10

Table 3.1: REFERENCES 11

REFERENCES	11
------------	----

3.2 Background

File integrity refers to the assurance that a file's content and associated metadata remain unchanged except through authorised, expected processes. In practice, a great many security incidents - ransomware encrypting data, malware replacing legitimate system binaries, an insider tampering with configuration files, or an attacker deleting log files to hide their tracks - announce themselves first as an unexpected change to a file. File Integrity Monitoring (FIM) is the security control designed to catch exactly this class of event: it establishes a trusted baseline of a system's critical files and directories, typically using cryptographic hashes, and continuously or periodically compares the current state of those files against that baseline to detect unauthorized modification, deletion or addition. FIM has a long history in computer security, tracing back to Kim and Spafford's Tripwire (1994), one of the earliest and most widely deployed file-integrity checkers, and to foundational work on intrusion detection more broadly. Today, FIM is not only a best-practice control but, in many sectors, a regulatory requirement: the Payment Card Industry Data Security Standard (PCI DSS) Requirement 11.5, for example, explicitly mandates the deployment of change-detection mechanisms to alert personnel to unauthorized modification of critical system files, configuration files or content files, and similar expectations appear in HIPAA and NIST SP 800-53 guidance.

3.3 Description of the Research Problem

Many small organisations and institutional environments - including academic and administrative systems typical of the Cameroonian context - either lack file integrity monitoring altogether or rely on manual, ad-hoc checks (spot-checking file timestamps, informal backups) that cannot reliably detect a deliberate, disguised modification. Where open-source FIM tools are available, their default configurations are typically either too coarse (missing changes between scheduled scans) or too noisy (flooding administrators with alerts for benign changes), and few provide a convenient way to review, approve or roll back detected changes. There is, accordingly, a need for a File Integrity Monitoring system that: (i) establishes a reliable cryptographic baseline of designated files and directories; (ii) detects unauthorized changes with minimal delay, ideally at or near real time; (iii) reduces false-positive alerts through a practical whitelisting/change-approval mechanism; and (iv) presents detected changes to an administrator through a clear, auditable interface. This proposal addresses that need through the design and implementation of such a system.

3.4 Research Questions

RQ3: How does a prototype implementation of the proposed system perform in terms of detection accuracy, detection latency, and false-positive rate when evaluated against a representative set of authorised and unauthorised file-change scenarios?

3.5 Objectives

General Objective: To design and implement a File Integrity Monitoring system capable of reliably detecting unauthorized changes to designated files and directories, with reduced false-positive alerting and a clear administrator-facing reporting mechanism.

To design a system architecture comprising a baseline/hashing engine, a change-detection monitor, a whitelisting/change-approval mechanism, and an administrator dashboard/alerting module. To implement a working prototype of the proposed system capable of monitoring a designated file set on a host system and detecting additions, deletions and modifications. To evaluate the prototype's detection accuracy, detection latency, and false-positive rate against a set of simulated authorised and unauthorised change scenarios.

1.4 Rationale

3.6 Justification and Motivation

The motivation for this study is the continued prevalence of file-tampering as an early or concealing step in security incidents, combined with the practical shortcomings - detection gaps, alert fatigue, and limited tamper-resistance - that persist even in mature, widely used FIM tools. A system that narrows these gaps while remaining lightweight enough for resource-constrained environments would offer tangible security value.

3.7 Significance

The study is significant to (i) system administrators and small organisations seeking a practical, low-overhead approach to detecting unauthorized file changes; (ii) the academic community, by contributing an evaluated design and prototype to the established body of file-integrity-monitoring research; and (iii) the candidate, as partial fulfilment of the requirements for the award of the M.Tech in Information Technology and Cybersecurity.

3.8 Scope

The study covers the design and prototype implementation of a File Integrity Monitoring system for a designated set of files and directories on a host system, including baseline generation, change detection, a basic change-approval/whitelisting mechanism, and an alerting/reporting interface. Evaluation will be conducted on simulated authorised and unauthorised change scenarios on a controlled test system.

3.9 Limitation

3.10 Conceptual Overview

File Integrity Monitoring is defined as a security process that continuously monitors and analyses the integrity of an organisation's files, configurations and related assets by comparing them against a trusted baseline in order to detect unauthorized changes or suspicious activity. The concept of systematically monitoring file usage for security purposes traces back to Anderson's (1980) foundational work on computer security threat monitoring, and was formalised for intrusion detection by Denning (1987), whose intrusion-detection model treats anomalous or unexpected system activity - including file modification - as evidence of a possible security violation.

3.11 Classical File Integrity Checking Approaches

The most influential early FIM tool is Tripwire, introduced by Kim and Spafford (1994), which established the now-standard approach of generating cryptographic checksums for a defined set of files, storing them in a signature database, and periodically comparing current file states against that database to flag unauthorized alterations. This checksum-baseline approach was subsequently adopted, with variations, by other open-source tools such as AIDE (Advanced Intrusion Detection Environment) and Samhain, which add features such as rootkit detection and centralised logging across multiple hosts. These tools remain widely deployed but are fundamentally scan-based: integrity is verified at scheduled intervals rather than continuously, leaving a window of exposure between scans.

3.12 Host-Based Intrusion Detection and Real-Time Monitoring

File-system integrity checking is recognised in the broader intrusion-detection literature as a distinct, valuable category of host-based intrusion detection system (HIDS), precisely because it retains visibility into stored data and executables even when other defences have been bypassed. Peddoju et al. (2020) survey the issues, challenges and solutions associated with FIM tools in depth, highlighting recurring problems: delayed detection where monitoring is not continuous, the administrative burden of maintaining and updating baseline databases, and vulnerability of the monitoring database and agent itself when an attacker gains local privileged access. Their work, and related studies on real-time and virtualization-assisted monitoring, motivate designs that move detection closer to real time - for example, using operating-system-level file-event notifications rather than relying solely on periodic re-scanning - and that isolate the baseline database from the monitored host where possible.

3.13 Challenges: False Positives and Tamper Resistance

A consistently reported operational challenge is the volume of false-positive alerts generated by routine, authorised changes such as software updates, log rotation or configuration management activity; without a mechanism to distinguish expected from unexpected change, administrators are prone to alert fatigue and may begin to ignore FIM notifications altogether, undermining the control's value. A second, related challenge is tamper resistance: because the FIM agent and its baseline database typically reside on the monitored host itself, an attacker with sufficient privileges can, in principle, modify the baseline to conceal their changes, motivating designs that store baselines externally, sign them cryptographically, or otherwise separate the monitoring function from the monitored system's trust boundary.

3.14 Compliance-Driven Requirements

Beyond its technical role, FIM is also shaped by regulatory and compliance frameworks. PCI DSS Requirement 11.5 explicitly mandates the deployment of change-detection mechanisms - such as file-integrity monitoring tools - to alert personnel to unauthorized modification of critical system files, configuration files or content files, and comparable expectations are found in HIPAA (for electronic protected health information) and NIST SP 800-53 controls, with NIST guidance on incident response additionally emphasising the evidentiary value of file hashes for forensic investigation. These requirements shape common design expectations for FIM systems: auditable logs, alerting on detected changes, and retention of evidence sufficient to support an investigation.

3.15 Research Gap

The literature confirms that the core cryptographic-baseline approach to file integrity checking is well established and effective in principle, but that practical deployments continue to be undermined by three persistent issues: detection latency from scan-based monitoring, false-positive-driven alert fatigue, and the tamper-exposure of on-host baseline data. Comparatively little published work combines a practical, lightweight solution to all three issues in a single system suited to small-scale or resource-constrained deployments, as opposed to enterprise-grade commercial FIM platforms. This proposal addresses that gap by designing and prototyping such a system.

3.16 Research Design

The study will adopt a Design Science Research (DSR) approach, appropriate for research whose principal contribution is a designed artefact - in this case, a File Integrity Monitoring system. The DSR cycle followed will comprise: (i) problem identification and objective definition (Chapter 1); (ii) design of the system architecture and baseline/detection strategy; (iii) implementation of a working prototype; (iv) demonstration of the prototype against simulated authorised and unauthorised change scenarios; and (v) evaluation against defined performance metrics, with findings feeding back into refinement of the design.

3.17 Proposed System Architecture Overview

The proposed architecture will be organised into four logical components: **Baseline Engine:** scans a designated set of files and directories, computes a cryptographic hash (SHA-256) and captures relevant metadata (size, permissions, timestamps) for each monitored item, and stores this as the trusted baseline in a database that is, where feasible, kept separate from the monitored host's own privilege domain. **Monitoring Engine:** continuously or near-continuously observes the monitored file set, using operating-system file-event notifications where available (to minimise detection latency) supplemented by periodic full re-scans as a fallback, and re-computes hashes for changed items to compare against the baseline. **Change-Classification and Alerting Layer:** compares each detected change against a configurable whitelist/change-approval list (e.g., an expected maintenance window, a known package-update signature) to suppress likely-authorised changes, and raises a prioritised alert - with the file path, timestamp, hash before/after, and (where available) the responsible process/user - for changes that are not recognised as authorised. **Administrator**

Dashboard: a web-based interface presenting a real-time feed of detected changes and alerts, a searchable change-history/audit log, and controls for reviewing, approving, or flagging an alert for follow-up investigation.

This modular design keeps the trust-sensitive baseline separate from the monitored host where deployment permits, and keeps the false-positive-suppression logic as a distinct, tunable layer rather than hard-coding it into the detection engine.

3.18 Baseline and Hashing Strategy

3.19 Tools and Technologies

Programming: Python, using standard cryptographic-hashing libraries (hashlib) and the watchdog library (or equivalent OS-level file-system-event API) for near-real-time change notification. Data storage: a lightweight database (e.g., SQLite/PostgreSQL) for the baseline and change-history/audit log.

Service architecture: a lightweight API framework (e.g., FastAPI/Flask) exposing the monitoring engine's status and alerts to the dashboard, containerised with Docker for portability.

Dashboard: a web-based front end for visualising alerts, the change-history log, and administrator review actions.

Version control and documentation: Git for source control and Markdown documentation of the design artefacts.

3.20 Detection and Alerting Logic

On each detected file-system event, the monitoring engine will re-hash the affected file, compare the new hash and metadata against the stored baseline, and classify the event as an addition, modification, or deletion. The change-classification layer will then check the event against the configurable whitelist (e.g., matching a known update-process signature or an approved maintenance window); unmatched events will be escalated as alerts with a severity level informed by the sensitivity of the affected file (for example, changes to authentication-related or system-configuration files will default to a higher severity than changes within a designated log directory).

3.21 Evaluation Metrics

The prototype will be evaluated using a test suite of simulated file-change scenarios comprising both authorised changes (e.g., a scripted software update) and unauthorized changes (e.g., a scripted unauthorized modification, deletion, or injection of content into a monitored file). Detection accuracy, false-positive rate (authorised changes incorrectly alerted) and false-negative rate (unauthorized changes missed) will be computed from these scenarios, alongside detection latency (time between the simulated change and the raised alert) measured under both event-driven and periodic-scan configurations for comparison.

3.22 Ethical Considerations

All testing will be conducted on a controlled, isolated test system created specifically for this study; no monitoring or change-simulation will be performed against live production systems or systems belonging to third parties without explicit authorisation. No personally identifiable or sensitive real-world data will be used in the test file set. This is consistent with the University's research-ethics expectations and the COLTECH guidance on academic integrity.

3.23 Proposed Work Plan

Table 3.2: Summary Table

Phase / Activity	Indicative Timeline
Literature review and requirements refinement	Month 1 - Month 2
Detailed system-architecture design	Month 2 - Month 3
Dataset preparation and prototype development (detection engines)	Month 3 - Month 5
Dashboard/reporting interface development and integration	Month 5 - Month 6
Testing, evaluation and performance benchmarking	Month 6 - Month 7
Dissertation write-up and internal review	Month 7 - Month 8
Submission and defense preparation	Month 8 - Month 9

REFERENCES

Anderson, J. P. (1980). Computer security threat monitoring and surveillance [White paper]. James P. Anderson Co.

Denning, D. E. (1987). An intrusion-detection model. *IEEE Transactions on Software Engineering*, 13(2), 222-232. <https://doi.org/10.1109/TSE.1987.232894>

Kim, G. H., & Spafford, E. H. (1994). The design and implementation of Tripwire: A file system integrity checker. In *Proceedings of the 2nd ACM Conference on Computer and Communications Security (CCS '94)* (pp. 18-29). ACM.

National Institute of Standards and Technology. (2006). *Guide to integrating forensic techniques into incident response (NIST Special Publication 800-86)*. U.S. Department of Commerce.

Peddoju, S. K., et al. (2020). File integrity monitoring tools: Issues, challenges, and solutions. *Concurrency and Computation: Practice and Experience*, 32(21), e5825. <https://doi.org/10.1002/cpe.5825>

PCI Security Standards Council. (2022). *Payment Card Industry Data Security Standard (PCI DSS), Requirement 11.5: Detect and respond to unauthorized changes*.